

1. Document Header and Versioning:

Project: Rookery - An Engine for Creating, Sharing, and Playing Chess Variants

Course: COMP SCI 4ZP6A, McMaster University, 2026-2027

Team number: 8

Version: Version 0.1 Baseline | Date: October 9, 2026

2. Personnel and Roles:

Team member	McMaster email	Primary discipline
Rafael Emir Nawaz	nawazr2@mcmaster.ca	Frontend engineering: variant editor; project management
Ibrahim Parvez	parvezi@mcmaster.ca	Frontend engineering: gameplay interface
Ramish Munir	munirr2@mcmaster.ca	Backend engineering: multiplayer sessions
Oliver Bahn	bahno@mcmaster.ca	Engine validation, simulation and testing
Fawz Mehfil	mehfilf@mcmaster.ca	Backend engineering and rules engine
Dilraj Bhandal	bhandald@mcmaster.ca	Backend engineering: multiplayer sessions
Chris Wang	wangz908@mcmaster.ca	Rules engine engineering

Rafael is assigned **project manager** and coordinates administrative work, document integration and deadlines. Fawz handles any misinterpretations of project implementations. All members share platform responsibilities, including authentication, persistence, publishing, discovery, deployment and CI/CD.

3. Definitions Acronyms and Abbreviations:

Term	Definition
Chess variant	A game defined by a custom board, pieces, rules and objectives.
No-code editor	Visual controls for defining supported rules without writing code.
Rules engine	Software that determines legal actions and updates game state.
Rule validator	Checks variant definitions for unsupported or conflicting rules.
Game state	Current board, pieces, turn, active effects and game outcome.
Remix	A modified variant that retains attribution to its source.
Authoritative server	The server validates actions and owns the accepted game state.
WebSocket	A persistent connection for exchanging live game updates.
CI/CD	Continuous integration and continuous delivery/deployment.

4. Contributions Table:

Team member	Sections authored or reviewed	Share
Rafael Emir Nawaz	Sections 1-6:	15%
Ibrahim Parvez	Sections 7-8:	16%
Ramish Munir	Sections 9-10:	17%
Oliver Bahn	Sections 11-12:	11%
Fawz Mehfil	Sections 13-14:	15%
Dilraj Bhandal	Sections 15-16:	15%
Chris Wang	Section 17:	11%

5. Executive Summary and Purpose:

Rookery aims to make creating and playing custom chess variants accessible to people without programming experience. A creator may want to change a board layout, introduce a new piece or replace the usual victory condition, but turning that idea into an online game requires more than describing its rules. Those rules must be represented consistently, legal moves must be enforced, and players must share the same game state. The project addresses this gap by bringing visual rule creation, validation and multiplayer play into one browser platform.

Users will create variants through visual tools for board configuration, piece placement, movement and capture rules, abilities, global modifiers and victory conditions. They will be able to test their designs, publish them to a community gallery and share links for multiplayer games. Other users will be able to discover published variants and remix them while preserving attribution. Game history and featured team-created variants will support and help users explore the platform.

The technical foundation will consist of a programmable rules engine, a rule validator, an authoritative multiplayer server and an automated simulation system. The engine will interpret supported custom rules and apply consistent state transitions. Validation will identify invalid or contradictory configurations, while simulations will help assess playability. The multiplayer server will validate player actions and synchronize accepted game updates between participants.

The intended impact is to reduce the effort needed to turn a chess-like game idea into a playable, shareable experience. Creators can experiment with rules, and players can explore the resulting designs. This SRS and Project Plan establishes a shared baseline for our team and course evaluators by documenting the intended users, system requirements and development approach. It will guide implementation and provide a basis for evaluating the completed platform.

6. Stakeholders and User Personas:

Stakeholder	Interest and expectations
Primary clients: creators and players	Create supported variants without coding, understand their rules and play them with other users.
Secondary: instructor and TAs	Assess technical depth, clear requirements, usability, architecture, automated testing and the feasibility of the project plan.

Secondary: development team	Use a shared requirements baseline to coordinate interfaces, implementation and verification.
Secondary: platform administrators	Manage published content and operate the service with appropriate moderation and monitoring tools.

Personas:

Variant creator: A chess enthusiast with little or no programming experience wants to change the board, piece behaviour or victory conditions. They need understandable editor controls, previews and validation feedback so they can test a design before publishing it.

Casual player: A user familiar with basic chess wants to try a variant with a friend. They need a straightforward invitation flow, a clear explanation of the variant and useful feedback about legal moves. Both players need a consistent game state.

Experienced designer and remixer: A user who experiments with chess rules wants to modify an existing variant and compare the results. They need expressive supported rule options, source attribution and simulation feedback that helps reveal playability problems.

Rookery is an original project conceived by our team. No external professor or industry partner is supervising the project or providing additional requirements.

7. Constraints & Assumptions:

Rookery will be a browser-based application. The editor will primarily target desktop browsers. No native application or GPU hardware is required.

The system will use Next.js and TypeScript, with a framework-independent rules engine shared by the browser, server, tests, and simulation tools. Variants will be stored as versioned data rather than executable user code. The rules engine must return the same result whenever it receives the same variant, game state, and action.

PostgreSQL will store users, variants, and games. A separate WebSocket-capable service will handle multiplayer. Redis may be added later if multiple multiplayer-server instances are required.

Initial deployment will use free, student, or course-provided hosting tiers. Vercel is the preferred web host, with a container host or McMaster CAS server for multiplayer. Paid services require team approval.

Rookery will collect only necessary account, variant, and game information. It will use HTTPS, secure sessions, authorization checks, input validation, and protected secrets. Users must be able to request access to or deletion of their account data. The team will follow applicable PIPEDA and GDPR privacy principles.

Team members will document their contributions and jointly approve any software licence or commercialization decision. Third-party libraries and assets must have compatible licences. The team will consult McMaster's Industry Liaison Office if questions about ownership or commercialization arise.

Baseline assumptions include:

- Games are deterministic, turn-based, and use perfect information.
- Published variant versions cannot be edited.
- Unsupported rule combinations may be rejected by the validator.
- No machine-learning model or training dataset is required.
- Online features require a stable internet connection.

8. Functional Requirements

Requirements are prioritized from P0 to P3. P0 defines the proof-of-concept system, P1 defines the planned final release, P2 contains optional improvements, and P3 contains stretch goals.

ID	Requirement	Input and Expected Output	Data Flow and Focus
P0-01	Users can create a variant by editing the board, pieces, movement rules, and objective.	Input: Editor selections. Output: Variant draft or validation errors.	Editor -> variant schema -> draft. Frontend.
P0-02	The rules engine validates actions and updates the game consistently.	Input: Variant, game state, and action. Output: Updated state or error.	Game state -> rules engine -> result. Backend.
P0-03	Users can immediately playtest a valid variant.	Input: Variant draft and player actions. Output: Legal moves, updated board, and game outcome.	Editor -> rules engine -> gameplay view. Full stack.
P1-01	Users can create accounts and save variant drafts.	Input: Account and draft information. Output: Stored or restored draft.	Client -> API -> PostgreSQL. Full stack.
P1-02	Users can publish, share, discover, and remix variants with attribution.	Input: Valid draft, publication details, or remix request. Output: Public variant page or attributed draft.	Client -> API -> database. Full stack.

ID	Requirement	Input and Expected Output	Data Flow and Focus
P1-03	Users can create and join online games with synchronized state and game history.	Input: Variant, invitation link, and player actions. Output: Confirmed game state, errors, and replayable history.	Client -> multiplayer server -> rules engine -> database. Backend.
P2-01	Creators can run automated simulations to identify possible playability problems.	Input: Variant and simulation settings. Output: Outcome and game-length statistics.	Request -> simulation worker -> report. Backend.
P2-02	Users can search community variants, while administrators can feature or moderate them.	Input: Search query or moderation action. Output: Search results or updated visibility.	Client -> API -> database. Full stack.
P3-01	The platform may support spectators, bot opponents, and collaborative editing.	Input: Game link, bot settings, or editing invitation. Output: Read-only game, bot action, or shared draft.	Client -> appropriate service -> engine or database. Full stack.
P3-02	The platform may support tournaments and player ratings.	Input: Players, variant, tournament settings, and results. Output: Pairings, standings, and ratings.	Tournament service -> games -> results. Backend.

Each GitHub issue and acceptance test will reference the relevant requirement ID. All P0 requirements must be completed for the proof of concept, while P1 requirements are targeted for the final release.

9. Data Strategy & Quantitative Metrics:

Metric	Description	Target Value
End-to-End Move Latency	Time from User A submitting a move to User B receiving the updated state.	≤ 100 ms

Metric	Description	Target Value
Socket Connection Concurrency	Minimum concurrent WebSocket connections supported per single backend instance without packet drop.	≥ 1000 active users
Connection Setup Time	The time required to establish a secure WSS connection and authenticate the user session before gameplay.	≤ 200 ms
Engine Execution Speed	Time required for the rules engine to validate an action and compute the subsequent state.	≤ 15 ms
Game Engine Accuracy	The percentage of game state transitions, legal move validations, and victory condition checks are correctly calculated by the engine according to the active variant's rules.	100 %
Page Load Time	The total time required for the web application to fully load, render all assets (HTML, CSS, JS), and become interactive.	≤ 3.0 s
First Contentful Paint (FCP)	The time it takes for the browser to render the initial interface / first piece of content from the DOM upon visiting the application.	≤ 1.5 s
State Hydration / Reconnection	Time to resynchronize game state when a player disconnects and re-enters the browser tab.	≤ 2.0 s
Rendering Frame Rate	Smoothness of piece drag-and-drop animations, board zooms, and move highlights	≥ 60 FPS, minimum 30 FPS
Database Query Latency	The execution time for standard PostgreSQL operations	Between 1ms to 50 ms

10. Non-Functional Requirements (NFRs):

10.1 Usability & Human-Computer Interaction

Client Environment & Compatibility: The application must run entirely within major versions of primary web browsers (Chrome, Firefox, Safari, Edge), requiring absolutely no native application installations or dedicated GPU hardware.

Device Responsiveness: The gameplay interface must be fully functional and responsive on desktop browsers. While some capabilities and functions can operate on mobile devices; it is not our intended audience.

No-Code Accessibility: The editor must allow users to configure board layouts, piece movements, and game rules entirely through visual interfaces, requiring absolutely no programming knowledge or script writing.

10.2 Performance, Scalability & Speed

Real-Time Synchronization: The multiplayer WebSocket service must transmit and synchronize player actions and game state updates in real-time with minimal latency as documented in section 9.

Engine Efficiency: The framework-independent rules engine must execute state transitions, move validations, and automated simulations quickly enough to prevent UI blocking in the browser and timeouts on the server.

Hosting Constraints & Scalability: The initial architecture must be optimized to operate within the resource limits of free, student, or course-provided hosting tiers (e.g., Vercel, basic container hosts), while retaining the architecture to scale horizontally (e.g., adding Redis) if capacity demands increase.

10.3 Security, Authentication & Data Privacy

Data Privacy Compliance: The system must comply with PIPEDA and GDPR principles, specifically requiring features that allow users to request access to, or complete deletion of, their account data (Right to be Forgotten). The platform must only collect and store the minimum necessary user account, variant, and game data required for functionality.

Security & Encryption: All data in transit must be encrypted using HTTPS/WSS. User sessions must be securely managed, and environmental secrets must be protected.

Input Validation & Authorization: Strict input validation must be enforced on both client and server sides to prevent malicious data entry. The server must enforce authorization checks before allowing variant publishing, editing, or account changes.

Execution Safety & Immunity: User created variants must be stored purely as versioned data (e.g. as JSON) rather than executable code to prevent malicious code injection or remote code execution via the rules engine.

10.4 Reliability, Fault Tolerance & Maintainability

Engine Determinism, Precision & Accuracy: The rules engine must be strictly deterministic; it must return the exact same resulting game state whenever provided with the same variant data, current game state, and player action.

Immutability & Integrity: Once a variant is published to the community gallery, that specific version's data must remain permanently immutable (uneditable) to preserve game history integrity and prevent breaking active multiplayer games or existing community remixes.

Architectural Maintainability & Adaptability: The rules engine must remain completely framework-independent so the exact same TypeScript code can be shared and executed across the Next.js browser client, backend server, automated tests, and simulation tools.

Authoritative Server Architecture: The multiplayer server must act as the ultimate authority on valid moves and game state. It must independently validate all player actions using the rules engine before broadcasting updates, preventing client-side desynchronization or cheating.

Graceful Rejection: The rule validator must be able to reliably identify and safely reject unsupported, invalid, or contradictory rule configurations during the variant creation process.

11. Risk Management & Feasibility Analysis

Potential Risk	Likelihood	Impact	Mitigation Strategy
Variant engine mechanic overcomplexity	High	High	Focus on integrating more critical engine features first, then gauge and adjust scale accordingly
Potential unplayable/broken variants	High	Medium	Add a validation system to make sure every variant has a win condition that can be met
Variant editor visual/UX overcomplexity	High	High	Categorize rules intuitively and provide guides and searchability features
Problems with integration between components	Medium	High	Establish all-encompassing framework plan prior to development of individual components
Team member falls behind on obligations	Low	Medium	Conduct weekly check-ins where team members share progress and request additional assistance if needed
Client synchronization issues	Medium	High	Server stores game state, users request to take actions which are validated by the server
Project scope is too small/complexity is too low	Low	Medium	Additional variant engine features are implemented/custom featured variants are developed

12. Team Communication & Meeting Cadence

The team will communicate via a dedicated SMS group chat as well as a Teams chat. Weekly 30-60 minute virtual meetings will be conducted using Teams, allowing each member to give progress updates, plan next steps, discuss technical/design aspects of development, and coordinate interconnectivity between project components. Each meeting will be given a

dedicated text file under /meetings in the project's Github page. This document will contain information pertaining to the meeting including:

- Date and time the meeting was held
- Duration of the meeting
- List of group members in attendance
- Meeting agenda (drafted prior to the meeting)
- Key points discussed during the meeting
- Actionable steps to be taken by team members following the meeting

Two virtual meetings will be held between team members and the professor:

1. Nov 2nd, 10:00am - 10:15am
2. Nov 16th, 11:00am - 11:15am

These meetings will be recorded on the project's Github page as with the weekly meetings, following the procedure outlined above. A meeting will also be held between team members and the TA in a similar fashion at a date and time mutually agreed on by team members.

13. Engineering Workflow & Version Control

13.1 Repository Structure

All code and documentation is kept in a single GitHub repository, so every component shares one CI pipeline, issue tracker and version history.

Directory	Contents	Primary owners
/src	Web application: variant editor, gameplay interface, gallery and accounts.	Rafael, Ibrahim
/packages/rules	Rules engine, variant schema and rule validator.	Fawz, Chris
/apps/realtime	Authoritative multiplayer server.	Ramish, Dilraj
/tests	End-to-end and simulation tests.	Oliver, with all
/docs	Architecture notes, deployment notes and SRS versions.	All
/meetings	Dated notes from sprint and TA meetings, with decisions and action items.	Rafael

13.2 Branching Strategy

The team follows GitHub Flow rather than GitFlow. Rookery is a continuously deployed web application with one production version, so GitFlow's develop, release and hotfix branches would add overhead without much benefit. The main branch is protected and always deployable. All work happens on short-lived branches created from main (for example, feature/42-piece-editor or fix/57-castling-bug) and merged back within a few days. Milestone deliverables are tagged on main so each demo can be reproduced.

13.3 Pull Request Review Rules

- Every change is merged through a pull request linked to its issue and needs at least one approving review. Rules engine and multiplayer changes must be reviewed by someone working in that area.

- All CI checks must pass before merging, and new game rules and bug fixes must include tests.
- Pull requests explain what changed and how it was tested, and are squash-merged into main.

13.4 CI/CD Pipeline Integration

GitHub Actions runs on every pull request and push to main. Each run performs linting, type checking, unit tests and a production build, with end-to-end tests covering the main user flows. Automated self-play simulations run nightly to catch rules engine errors. Every pull request receives a preview deployment, and main deploys to production automatically once all checks pass.

13.5 Issue Tracking

Work is tracked in GitHub Issues on a GitHub Projects board (Backlog, Ready, In Progress, In Review, Done). Features are written as user stories with acceptance criteria, labelled by area and priority, estimated in story points, and grouped into milestones that match course deliverables.

13.6 Agile / Scrum Framework

Element	Practice
Sprint length	Two weeks, giving about six sprints per term.
Roles	Rafael acts as Scrum Master and Fawz as Product Owner; all members form the development team.
Meetings	Sprint planning at the start, written stand-ups three times a week with a weekly meeting, and a review and retrospective at the end.
Backlog	Built from the SRS requirements and prioritized by Rafael, starting with the rules engine, then core flows (editor, publishing, multiplayer), then enhancements. Refined mid-sprint.
Definition of Done	Merged through an approved pull request with passing CI, tests and met acceptance criteria.

14. Infrastructure & Compute Resources

Rookery is a browser-based web platform. GPU clusters are not required, since the rules engine and simulations are CPU-based rather than machine learning, and no App Store or Google Play accounts are needed, since the application is a responsive website.

Environment	Resources
Local	Node.js 22 and Docker Compose on team members' machines.
CI	GitHub Actions hosted runners for checks, tests and simulations.
Preview	A Vercel deployment for each pull request, using a separate test database.
Production	Vercel for the web application, a container host such as Fly.io for the multiplayer server, and managed PostgreSQL for data.

McMaster CAS virtual servers serve as a fallback for hosting the multiplayer server or running large simulation batches if free hosting tiers are not enough. Since the application runs in Docker, it can move there without code changes. All services are used within free or student tiers, and credentials are never committed to the repository.

15. Technology Stack & Architectural Rationale

Rookery will be written mainly in TypeScript. Section 7 requires that a given variant, game state and action always produce the same result, whether the rules engine runs in the browser, on the multiplayer server, in tests or in simulations. TypeScript runs natively in all four places, so the engine only has to be written and tested once. Its static types also catch mistakes in variant data, which has many nested parts, before the code runs.

The rules engine (/packages/rules) is a TypeScript library with no framework code. It takes a variant, a game state and an action, and returns the next state. The web application (/src) imports it for editor previews and local playtesting. The multiplayer server (/apps/realtime) imports the same package and checks every move with it before sending the move to the other player. That makes it the authoritative server, so both players always see the same board.

Most of the remaining choices follow from two decisions: using TypeScript everywhere and hosting the web application on Vercel.

Choice	Rationale
Next.js (React)	The editor is made of many repeated parts (board, squares, pieces, rule cards), and React's components fit that well. React also has more third-party libraries than any other UI framework. Next.js adds page routing, server-rendered pages for shared links, and API routes for accounts and publishing. It deploys to Vercel without extra setup
Node.js 22 with Socket.IO	Vercel's serverless functions cannot hold the long-lived connections a live game needs, so multiplayer runs as its own container service. Socket.IO provides game rooms and automatic reconnection, which we would otherwise have to build ourselves on raw WebSockets. If more than one server instance is needed later, Socket.IO's Redis adapter supports the Redis option in Section 7.
PostgreSQL	Rookery's data is relational. Users own variants, a remix points to the variant it came from, and each game records the variant version it used. PostgreSQL enforces these links and can store each variant definition in a JSONB column. Prisma gives us typed database queries and handles schema migrations.

Rookery does not use machine learning. The rule validator checks variant definitions directly and the simulator runs automated self-play games. Both work on a brand-new variant without any training data.

Testing focuses on the rules engine, since every other part of the system depends on it.

Level	Tool	Purpose
Unit and engine	Vitest	Tests engine, validator and server functions. Standard chess, written as a variant, must match the given performance test move counts.

Property-based	fast-check	Generates random positions to check that the engine gives the same result for the same input and never allows an illegal move.
Integration	Vitest, Socket.IO client	Runs multiplayer sessions against a test database, including rejected illegal moves and reconnecting after a dropped connection.
End-to-end	Playwright	Runs the main user flows in real browsers on every pull request.
Simulation	Self-play scripts	Plays batches of games to catch crashes and victory conditions that can never be reached.

16. Proof of Concept Demonstration Plan

The November 23, 2026 proof of concept will demonstrate the core path from creating a rule to playing an enforced game.

The demonstration will include:

1. Opening a prepared variant in the editor.
2. Changing the board or a piece's movement.
3. Showing validation errors for an invalid configuration.
4. Displaying legal moves and rejecting an illegal move.
5. Completing a short sequence that triggers a victory condition.
6. Exporting and re-importing the variant as versioned JSON.

The PoC will be successful if the same variant and action sequence always produce the same result, invalid rules are reported without crashing, and legal moves are generated within 100 milliseconds for the demonstration board.

Fawz will own the variant schema, rules engine, validator, and related tests. Dilraj will co-own application integration, deployment, and the live demonstration.

Development schedule:

- **October 5-25:** Define the schema and implement the basic rules engine.
- **October 26-November 8:** Connect the editor, validator, and local playtest.
- **November 9-15:** Add JSON import/export and integration tests.
- **November 16-22:** Fix issues, rehearse, and prepare a backup recording.
- **November 23:** Present the working PoC and record evaluator feedback.

Authentication, publishing, gallery features, database persistence, and online multiplayer are deferred until after the PoC.

17. Project Schedule & Gantt Chart:

The Rookery development schedule spans Fall 2026 through the April 2027 Capstone Expo. Development is organized around the course milestones and two-week development sprints. Fall 2026 focuses on system architecture, the rules engine, the no-code variant editor, and the Proof of Concept. Winter 2027 focuses on multiplayer functionality, community features, system integration, verification and validation, deployment, and final Expo preparation.

Project Gantt Chart

 Rookery_Part17_Gantt_2026-2027_chris

